Subreddit Post Sentiment Classification

Rajat Choudhary:2604962C

Introduction

Reddit is made of threads that contain posts generated by the users. A similar 'real-world' collection of Reddit post is provided where the information from these threads is extracted according to different features such as the title of the post, URL, etc with the addition of sentiment. This report comprises the discussion of classifying these subreddit posts into different sentiments classes by analysing the body text of the posts.

Exploring dataset

Our Dataset Skeleton comprises training, validation and test datasets, where the training, validation and test data is split into a ratio of 63:16:21. This may prove to an advantageous split as the model can be well trained, validated and tested on this kind of data ratio. However, further exploration indicates a class imbalance where the class percentages range from as low as 0.2% for 'very negative' labels to 63% for 'neutral' classes in one of the sets. This may lead to the model becoming biased towards the high-frequency class and having a negligible or low probability for the prediction of low-frequency classes. (i.i)

Classifier performances

Six Different Classifiers, namely, Dummy Classifier - Most frequent (DCMF), Dummy Classifier- Stratified (DCS), Logistic Regression-One-hot vectorization (LROH), Logistic Regression-TF-IDF vectorization (LRT), Support Vector Classifier (SVC) and one classifier of our choice were used to fit on the label encoded training data. Intrinsic tokenizers of both the vectorizers(TF-IDF and One-Hot) are faster than any external methods of tokenisation and that's why the tokenizers are implemented intrinsically. After classification on train and test data, the

different metric scores, Macro-precision(MP), Weighted-precision(WP), Macro-Recall(MR), Weighted-Recall(WR), Accuracy(ACC), Macro-F1(M_F1), Weighted-F1(W_F1) are measured. Weighted-F1(W_F1) is taken as the definitive measure whilst comparing with other classifiers. Analysis of the results proves that LROH gives the best metrics while both the Dummy Classifiers proves to be the least scoring classifiers. The reason for the low scores of the Dummy Classifiers is that they predict solely based on the distribution of labels in the true classes and LROH performs the best, as the order of text doesn't necessarily determine the sentiment of a text. Also, higher metric scores are observed on the training set as it the same dataset that the classifier is fitted. (i.ii)

Facebook fastText(FT) is known for being extremely fast and reliable for many text classification problems and because of its smooth execution on a typically large dataset with efficient scores, it was chosen as the additional classifier over other classifiers, forming a very efficient classifier for large datasets. Additionally, it also has its intrinsic vectorizer which was chosen.

	DCMF	DCS	LROH	LRT	SVC	FT	Best
MP_Train	0.126528	0.193212	0.983331	0.916341	0.750607	0.763173	LROH
WP_Train	0.400235	0.470599	0.966154	0.835100	0.869245	0.965219	LROH
MR_Train	0.200000	0.193078	0.851584	0.409262	0.424704	0.734364	LROH
WR_Train	0.632641	0.472648	0.965398	0.817103	0.862416	0.973307	FT
Acc_Train	0.632641	0.472648	0.965398	0.817103	0.862416	0.973307	FT
M_F1_Train	0.154998	0.193125	0.907058	0.461072	0.457400	0.747740	LROH
W_F1_Train	0.490291	0.471613	0.964656	0.789095	0.837836	0.969066	FT
MP_Test	0.125199	0.203134	0.642767	0.578100	0.457835	0.512551	LROH
WP_Test	0.391871	0.473256	0.727408	0.728729	0.717855	0.707983	LRT
MR_Test	0.200000	0.202614	0.422725	0.325215	0.287530	0.333434	LROH
WR_Test	0.625996	0.475847	0.743277	0.738795	0.728088	0.733815	LROH
Acc_Test	0.625996	0.475847	0.743277	0.738795	0.728088	0.733815	LROH
M_F1_Test	0.153997	0.202840	0.468387	0.348681	0.286049	0.356149	LROH
W_F1_Test	0.482007	0.474539	0.727340	0.699172	0.675634	0.700007	LROH

As demonstrated by the results, fastText proves to be an efficient classifier as it incorporates a very low time complexity and gives higher metric results than many other classifiers, but it falls short of LROH due to requiring hyper-parameter tuning and improvement in the imbalanced dataset. (i.iii)

Parameter Optimisation

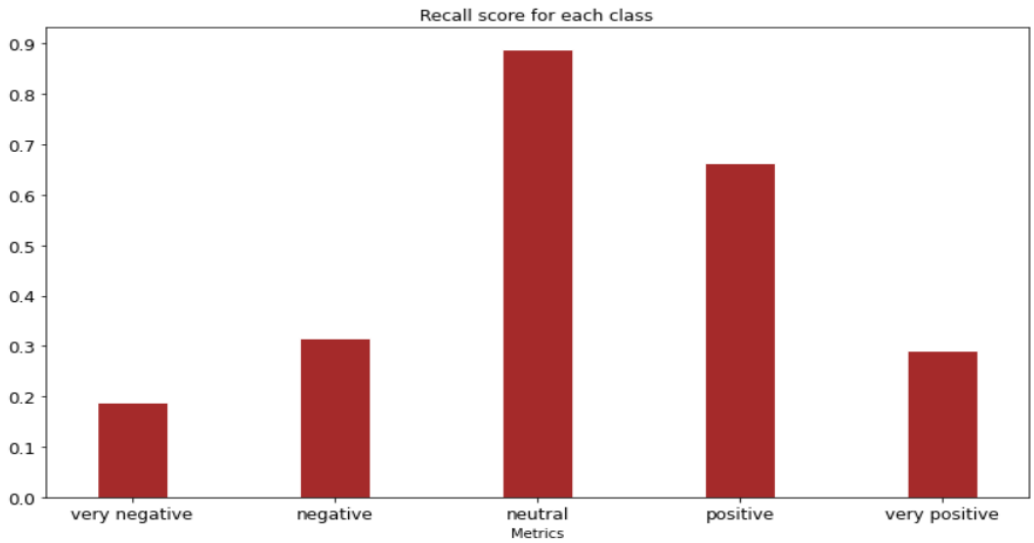
Parameter optimisation is performed to improve the performance of our baseline classifier (LRT). The validation dataset contains a lesser amount of data and may subject to issues such as underfitting. To avoid this, validation data is combined with the training data. The increased dataset size makes it easier for the model to fit. The described parameters (Regularized C values:10, sublinear_tf: True, max_features:5000) are found by sequentially traversing through the values by Grid Search. The scoring method used in Grid Search CV was taken to be "weighted_F1(W_F1)" as in sentiment analysis problems, we need to reduce both the precision and recall. The final parameter to tune was selected as 'solver' for the classifier, as it penalises large parameter values and implements various loss functions that prove to be good at evaluating our baseline classifier. 'Saga' solver proves to be the most appropriate solver as it supports the 'elasticnet' penalty. (ii.i)

After the optimisation of parameter values, the classification report and confusion matrix of our model is evaluated.

```
Evaluation for:  LRIDF(Parameters Tuned) Validation Test
Acc = 0.768 P = 0.664 R = 0.469 F1= 0.523
              precision    recall  f1-score   support

           0       0.67      0.19      0.29        32
           1       0.57      0.32      0.41       282
           2       0.80      0.89      0.84      2514
           3       0.73      0.66      0.69      1102
           4       0.56      0.29      0.38        86

    accuracy                           0.77      4016
   macro avg       0.66      0.47      0.52      4016
weighted avg       0.76      0.77      0.76      4016

Confusion Matrix :
  [[   6    9   17    0    0]
 [   3   89  183    6    1]
 [   0   56 2232  221    5]
 [   0    1  356  731   14]
 [   0    0   16   45   25]]
```

After parameter optimisation, we can observe that although the weighted f1 has increased from 0.73 to 0.76, there are very low recall scores of certain classes, this accounts for a huge number

of False Negatives which can prove detrimental to our model. These shortcomings occur when there is a huge bias between how the classes are distributed in the data, and such a model becomes prone to predicting False Negatives. The disparity is visible in the visualisation below. (ii.ii)



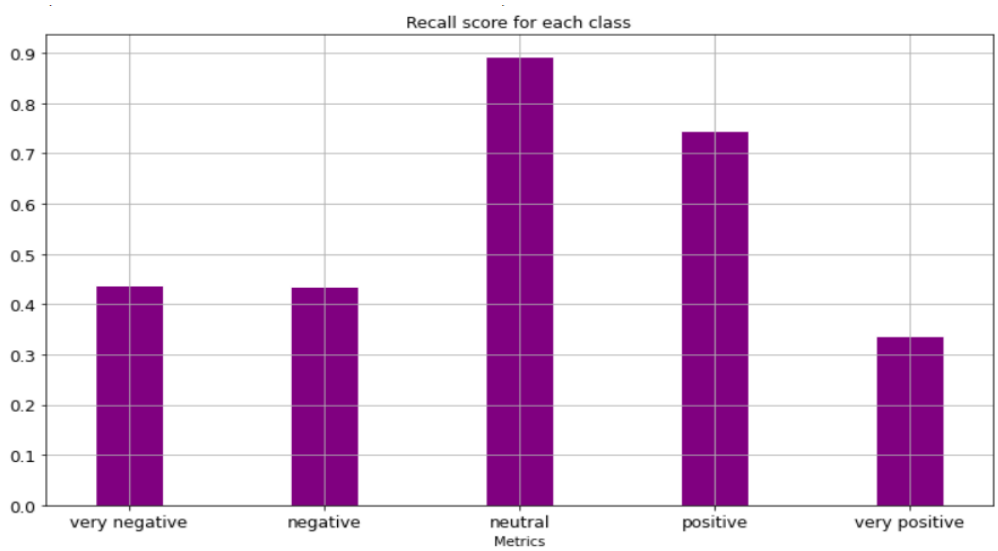
Feature Addition

The characteristic that closely determines a sentiment of a particular class is the amount of subjectivity it contains. The text can be written with a strong amount of subjectivity yet the model may be classifying it as a neutral comment since it doesn't train on that feature. Hence the amount of subjectivity a text contains becomes an important criterion in training a model and the first feature that got chosen is 'sentiment.subjectivity'. The second important characteristic that plays a part in sentiment analysis is the type of post in which a text may be written. A text may be a humourous comment and our model may categorize it as a negative reaction. Hence, the second feature that we add to our model, is 'majority_type'. (iii. i)

The baseline LRT model with tuned parameters along with the addition of features and trained on the combined model of train and validation. Then this model is used to predict the test data and the performance summary is evaluated, and the final results of the model can be tabularized. (iii. ii)

Test Data Statistics After Feature Addition	
Macro precision	0.675685
Weighted Precision	0.797067
Macro Recall	0.569800
Weighted Recall	0.804532
Accuracy	0.804532
Macro F1	0.611443
Weighted F1	0.799006

All the scoring metrics are increased when compared to the earlier model. Especially weighted_f1 score has also increased from 0.73 to 0.799. Further analysing the error from above, new recall scores of each class are visualized.



It can be concluded that even a heavily low data percentage class, 'very negative' has a recall score of greater than 0.4%, the False negatives predicted on this class have been severely reduced as compared to before. Also, the weighted_f1 and accuracy scores are high which makes it a more stable model and this attributes to the analysis of our dataset and which feature has the most effect. However, the model could be improved if the dataset is applied to a more efficient preprocessing of the text body where the text is removed of the contained URLs etc. (iii. iii)